HERMES · PHASE 0 · LESSON 0002 · LAB EXERCISE 01-RAW-HTTP · ~45–60 MIN

Run the Raw Request, Against a Mock

You send lesson 0001's raw request against a local mock. Then you break it six ways, one responsibility at a time.

Why this matters for Hermes

Hermes is tested against fakes and recorded fixtures before it makes a paid API call. This lesson builds the first such stand-in: a local server that speaks the Messages API's rules with no model behind it. Scenario step S4 (model calls) sends this same request from inside Hermes. The six-responsibilities thread continues here: each break in the lab makes one responsibility fail in front of you.

1 · The lab you just gained

A new folder sits next to `lessons/`: `hermes-sdk-lab/`, a pnpm workspace that holds every lab exercise. One install at the root serves all of them. The configuration files carry Phase 0 material of their own.

```
hermes-sdk-lab/
├── pnpm-workspace.yaml    one install serves every exercise ("0*")
├── package.json           shared toolchain: typescript, tsx, @types/node
├── pnpm-lock.yaml         the lockfile (see below)
├── tsconfig.base.json     the strict config every exercise extends
└── 01-raw-http/
    ├── package.json       "type": "module" plus the scripts you run
    ├── tsconfig.json      extends ../tsconfig.base.json
    ├── fixtures/message-response.json   the canned response, real shape
    └── src/
        ├── mock-server.ts provided. It enforces the gates in Diagram 1
        └── client.ts      yours. The TODOs are the exercise
```

Three configuration ideas carry the whole lab.

`pnpm-lock.yaml` is a *lockfile*: the record of the exact version every dependency resolved to, transitive dependencies included. `package.json` only states ranges. The range `"typescript": "^5.7.0"` accepts any 5.x from 5.7 up, and the lockfile records that the install resolved it to 5.9.3. Commit the lockfile, and every later install reproduces the identical tree.

The line `"type": "module"` switches the package to *ES modules*: JavaScript's standard module system, `import` and `export` with fixed, analyzable meaning. Node's older CommonJS system resolves `require` calls at runtime. The tsconfig option `module: "nodenext"` makes TypeScript resolve imports the way Node will. The compiler and the runtime then agree about every import.

The shared `tsconfig.base.json` turns on *strict mode*: the TypeScript compiler flags that decide what a compile-time claim may hide. Each line tightens one kind of claim.

tsconfig line	What it refuses to let a claim hide
`strict: true`	the umbrella flag. No implicit `any`, and `null` and `undefined` become real types your code must handle.
`noUncheckedIndexedAccess`	an index can miss at runtime, so `content[0]` gains `undefined` in its type. You meet this when you print the first content block.
`exactOptionalPropertyTypes`	an omitted property and a property set to `undefined` are different things. JSON on the wire has no `undefined`, so the types keep them apart.
`verbatimModuleSyntax`	a type-only import must say `import type`, because the type will not exist at runtime. Lesson 0001 named this fact type erasure.
`noEmit: true`	nothing. `tsx` runs the `.ts` files directly, and `tsc` only checks them. Types are erased either way.

2 · A test double for the wire

`src/mock-server.ts` is a *test double*: a stand-in that answers the same calls as a real dependency, so the code under test cannot tell the difference. This double stands in for the Messages API. It enforces the checks lesson 0001 circled, and it returns either a canned `Message` with the real response shape or a real error envelope. The circled numbers below are lesson 0001's six responsibilities: ① endpoint, ② authentication, ③ API version, ④ request and response shape, ⑤ errors, ⑥ cancellation.

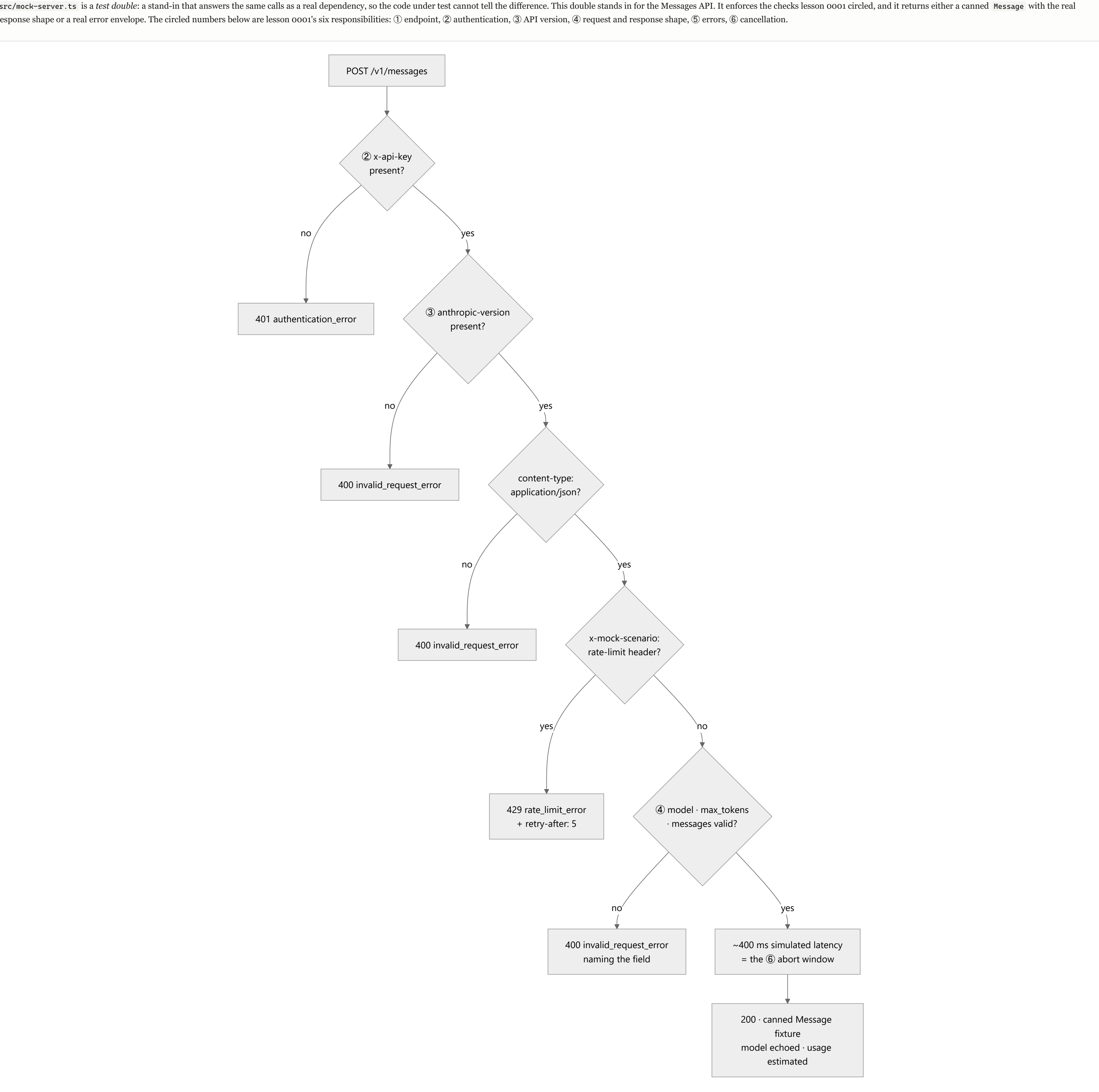
POST /v1/messages
② x-api-key present?
no
401 authentication_error
yes
③ anthropic-version present?
no
400 invalid_request_error
yes
content-type: application/json?
no
400 invalid_request_error
yes
x-mock-scenario: rate-limit header?
yes
429 rate_limit_error + retry-after: 5
no
④ model · max_tokens · messages valid?
no
400 invalid_request_error naming the field
yes
~400 ms simulated latency = the ⑥ abort window
200 · canned Message fixture model echoed · usage estimated

Diagram 1. Every gate a request passes through in `mock-server.ts`, in source order. Each error exit uses the real envelope shape.

Here is one full exchange, captured against the running mock. The request carries responsibilities ① to ④ as bytes on the wire.

```
POST /v1/messages HTTP/1.1                    ① the endpoint
host: localhost:8787
content-type: application/json
x-api-key: any-value-passes                   ② checked for presence only
anthropic-version: 2023-06-01                 ③ pins one revision of the rules

{"model":"claude-opus-4-8","max_tokens":1024,  ④ the request shape
 "messages":[{"role":"user","content":"What is an SDK?"}]}

HTTP/1.1 200 OK                               after ~400 ms of simulated latency
content-type: application/json
request-id: req_mock_0001

{
  "id": "msg_mock_0001",
  "type": "message",
  "role": "assistant",
  "model": "claude-opus-4-8",                 the model you sent, echoed back
  "content": [{ "type": "text", "text": "Hello from the mock.  ..." }],
  "stop_reason": "end_turn",
  "stop_sequence": null,
  "usage": { "input_tokens": 12, "output_tokens": 42,  ... }
}
```

A failed gate answers with the same envelope shape the real API uses. This is what your client reads when the key header is missing.

```
HTTP/1.1 401 Unauthorized
request-id: req_mock_0002

{ "type": "error",
  "error": { "type": "authentication_error",
             "message": "x-api-key header is required" },
  "request_id": "req_mock_0002" }
```

The double checks presence, not truth. Any value in `x-api-key` passes, because only Anthropic's servers know which keys are real. A double can enforce the shape of the rules. It can never verify the truths behind them: real keys, real rate limits, the real model catalog.

The request above reached `localhost:8787` because the *base URL* is the host part of the request's address, treated as configuration the caller may change. The API fixes the path `/v1/messages`, never the host. Your client reads `ANTHROPIC_BASE_URL` and falls back to `localhost:8787`. The official SDK exposes the same option as `baseURL`. Exercise 02 aims the real `@anthropic-ai/sdk` at this same mock through it, with no change to SDK code.

New today.

Five ideas are new.

- A test double enforces shape, never the truths behind the shape.
- The base URL turns the request's destination into configuration.
- A lockfile records what an install resolved.
- Compiler configuration decides what a compile-time claim may hide.
- Failures arrive as an HTTP status plus a JSON error envelope.

Reprise: the six responsibilities, and the untyped body of lesson 0001's raw request.

3 · The exercise

The full commands live in `hermes-sdk-lab/01-raw-http/README.md`. The exercise has five steps.

1. Install the toolchain: `pnpm install` at the lab root. Then find `typescript` in `pnpm-lock.yaml`. That is §1's range, resolved and recorded.
2. Start the mock in a first terminal: `pnpm mock` inside `01-raw-http/`.
3. Write the request: fill in `src/client.ts` from memory. The six responsibilities are the checklist, and the mock's errors are the feedback. Run `pnpm request` in a second terminal until you see a 200 with `stop_reason`, the text block, and `usage`.
4. Break the request six ways. Change one thing at a time. Watch both terminals on every run.

Break	What you observe
Delete the `x-api-key` header	`401 authentication_error`. Responsibility ② is a header the wire demands.
Delete the `anthropic-version` header	`400 invalid_request_error`. Responsibility ③ pins which revision of the rules you speak.
Delete `max_tokens` from the body	a 400 naming the field (④). Nothing before the server caught it.
Misspell `messages` in the body	a 400 at runtime. The SDK's types caught this same typo at compile time in lesson 0001.
Add the header `x-mock-scenario: rate-limit`	a 429 with a `retry-after` header, the input a ⑤ retry policy reads.
`setTimeout(() => controller.abort(), 100)`	an `AbortError` in the client, and the mock logs an abort mid-flight (⑥).

5. Run `pnpm typecheck`. Count which of the six failures TypeScript could have caught. Almost none, because you built the body untyped. Exercise 02 shows which ones the SDK moves to compile time.

4 · Exercise: who can catch it?

Classify each mistake

Answer from memory before you click. Feedback is immediate.

The request body omits `max_tokens`.

The mock can catch it | Only the real API can catch it

The `x-api-key` value is not a real key.

The mock can catch it | Only the real API can catch it

The `x-api-key` header is missing entirely.

The mock can catch it | Only the real API can catch it

The model id is misspelled: `claude-opsu-4-8`.

The mock can catch it | Only the real API can catch it

You write `mesages` instead of `messages` in the body.

The mock can catch it | Only the real API can catch it

Your per-minute rate limit is exhausted.

The mock can catch it | Only the real API can catch it

The `content-type` header is missing.

The mock can catch it | Only the real API can catch it

Score: 0 / 0

5 · Quiz

Four questions

Q1. Exercise 02 will point the official SDK at this same mock. What makes that possible without touching the SDK's code?

- The SDK exposes a base-URL override option
- The SDK detects mocks on localhost by port
- The mock intercepts the SDK's DNS lookups
- The SDK ships with a built-in mock server

Q2. You commit `pnpm-lock.yaml`. A teammate runs `pnpm install` next month. What does the lockfile guarantee they get?

- They get the newest compatible versions
- They get the exact versions you resolved
- They get versions vetted by the registry
- They get versions cached on your machine

Q3. Under `noUncheckedIndexedAccess`, what is the type of `data.content[0]`?

- The element type, exactly as declared
- never, because the index may not exist
- The element type joined with undefined
- unknown, until you narrow it yourself

Q4. The mock waits ~400 ms before every 200 response. What is that delay for?

- It simulates rate limiting on every request
- It gives the abort signal a window to interrupt
- It mimics DNS resolution for localhost calls
- It batches concurrent requests before replying

Score: 0 / 0

6 · Say it in your own words

After the lab, answer these four in the chat, from memory.

1. Define a test double in one sentence. Name the one kind of thing it can never verify.
2. What does mock-first development buy, beyond saving API costs?
3. What is the base URL, and how will exercise 02 use it?
4. Name two mistakes this raw client only found at runtime that lesson 0001's SDK version caught at compile time. Say what the SDK adds that makes the difference.

What this lesson names, and where it is built

Capability	Status	Where
the official SDK pointed at this mock	implemented	lesson 0003, lab exercise 02
runtime validation of the response bytes	implemented	lesson 0005, lab exercise 04
a fake behind a Hermes-owned interface (`FakeModelGateway`)	implemented	lesson 0006, lab exercise 05

If you keep only four sentences, keep these. A test double answers the same calls as the real dependency and enforces shape, never the truths behind it. The base URL is configuration, so one variable points the same request at a mock or at the real API. You built the body untyped, so the server was the first check it ever met. Every failure arrived as an HTTP status plus a JSON error envelope naming a type and a message.

Primary source

Read next: the Messages API reference, after the exercise. Check the mock against it: which required fields, headers, and error shapes did the double get right, and which did it leave out? For the cancellation side, read MDN's AbortController page (~5 min).

Verified against typescript 5.9.3, tsx 4.23.1, Node 22.14.0 and pnpm 11.17.0, as resolved in `pnpm-lock.yaml`. The wire blocks above are real exchanges against the mock, captured 2026-08-17. The mock's response and error shapes follow the Messages API. Its error wording is an approximation. This lesson's terms are registered under `wiki/terms/`.

Next: lesson 0003 points the official `@anthropic-ai/sdk` at this very mock and watches it absorb the six responsibilities.